

UNFISH: Technical Architecture & Systems Engineering Specification

Comprehensive System Architecture, Virtual Machines, Compilers & Systems Programming

The Unfish Core Engineering Team

October 2026 - Version 2.1.0

Contents

1	1. System Architecture Overview & Engineering Contract	1
1.0.1	1.1. Core Engineering Invariants	1
2	2. Lexical & Syntactic Analysis	2
2.0.1	2.1. Indentation Stack Algorithm (src/lexer/uf_lexer.c)	2
2.0.2	2.2. Pratt Operator Precedence Parser (src/parser/uf_parser.c)	2
3	3. Virtual Machine Architectures: Stack vs. Register	3
3.0.1	3.1. Tier 2: 57-Opcode Stack-Based Virtual Machine (src/vm/uf_vm.c)	3
3.0.2	3.2. Tier 3: 256-Register Computed-Goto Virtual Machine (src/vm2/uf_regvm.c) . .	4
4	4. Memory Management: Linear Arenas & Mark-Sweep GC	4
4.0.1	4.1. Compilation Linear Arenas (src/common/uf_arena.c)	4
4.0.2	4.2. Runtime Garbage Collection (src/runtime/uf_runtime.c)	4
5	5. Structured Exception Handling & Unwinding Engine	4
5.0.1	5.1. The Dual-Stack Synchronization Invariant	5
6	6. Systems Programming & Low-Level Buffers	5
6.0.1	6.1. Buffer Capabilities	5
7	7. Ahead-of-Time Native C99 Transpiler & WebAssembly	5
7.0.1	7.1. C99 Code Generation (src/codegen/uf_emit_c.c)	5
7.0.2	7.2. WebAssembly Target (--wasm)	5
7.0.3	7.3. Bare-Metal Embedded Target (--embedded --arm)	6
8	8. Verification & Differential Parity Harness	6

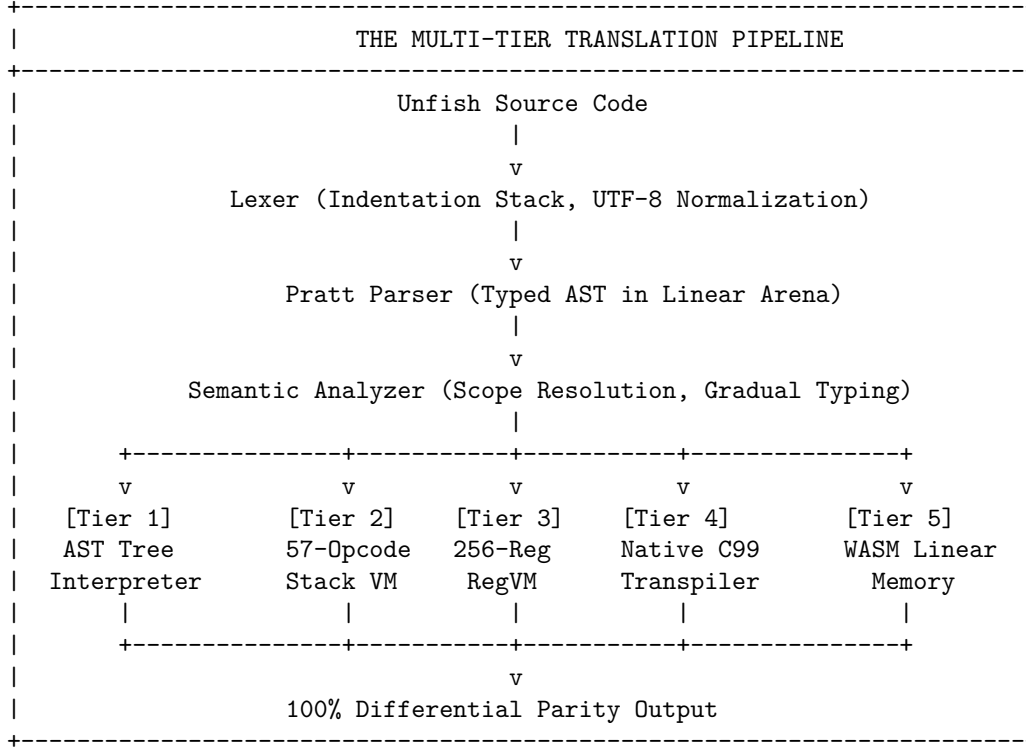
1 1. System Architecture Overview & Engineering Contract

Unfish is a general-purpose programming language, multi-tier virtual machine runtime, optimizing compiler, and systems programming platform implemented in pure ANSI C99 with **zero external dependencies**.

1.0.1 1.1. Core Engineering Invariants

1. **ANSI C99 Standard Conformance:** Every source file in `src/` compiles under any standard C99 compiler (`gcc -std=c99`, `clang -std=c99`, `tcc`, `MSVC`) linking exclusively against the standard C

- library (`libc`) and math library (`libm`).
2. **5-Way Differential Parity:** Every program evaluated by the five backends (AST Interpreter, Stack VM, Register VM, Native C99, WebAssembly) must produce identical `stdout`, `stderr`, and exit codes.
 3. **Deterministic Memory Safety:** Compilation memory is managed via linear bump arenas with $O(1)$ batch destruction. Runtime memory is managed via a cycle-safe mark-and-sweep garbage collector verified with zero leaks under LLVM AddressSanitizer (ASan) and LeakSanitizer (LSan).
 4. **Single-Binary Zero-Bloat Deployment:** The entire language platform—including compilers, virtual machines, code formatters, language server (LSP 3.17), and standard library—compiles into a single **680 KB static executable**.



2. Lexical & Syntactic Analysis

2.0.1 2.1. Indentation Stack Algorithm (`src/lexer/uf_lexer.c`)

Unfish uses indentation to define block scope without curly braces or semicolons. The lexer maintains an explicit column stack:

$$\text{Stack} = [S_0, S_1, \dots, S_k], \quad S_0 = 0$$

For each non-blank line: 1. Column count C of leading whitespace is measured. 2. If $C > S_k$: Push C , emit `INDENT`. 3. If $C == S_k$: No token emitted. 4. If $C < S_k$: Repeatedly pop until $S_j == C$, emitting `DEDENT` for each popped level. If no match is found, an `IndentationError` diagnostic with 2-D coordinates is generated. 5. At EOF: Pop all remaining levels down to 0, emitting matching `DEDENT` tokens followed by EOF.

2.0.2 2.2. Pratt Operator Precedence Parser (`src/parser/uf_parser.c`)

Expressions are parsed using Vaughan Pratt’s top-down operator precedence algorithm across 13 distinct precedence tiers:

Precedence	Level Name	Operators	Associativity	Description
1 (Lowest)	Pipe	<code>\ ></code>	Left	Forward pipeline data flow
2	Logical OR	<code>or</code>	Left	Short-circuit disjunction
3	Logical AND	<code>and</code>	Left	Short-circuit conjunction
4	Equality	<code>==, !=</code>	None	Structural equality
5	Relational	<code><, <=, >, >=</code>	None	Ordered comparison
6	Bitwise OR	<code>bor(...)</code>	Left	32-bit bitwise OR
7	Bitwise XOR	<code>bxor(...)</code>	Left	32-bit bitwise XOR
8	Bitwise AND	<code>band(...)</code>	Left	32-bit bitwise AND
9	Bit Shifts	<code>shl, shr, sar</code>	Left	Logical/arithmetic shifts
10	Additive	<code>+, -</code>	Left	Addition, subtraction
11	Multiplicative	<code>*, /, %</code>	Left	Multiplication, division, modulo
12	Unary Prefix	<code>-, not, bnot, await</code>	Right	Prefix negation
13 (Highest)	Primary & Call	<code>() , [] , . , f"..."</code>	Left	Invocations, indexing, member access

3. Virtual Machine Architectures: Stack vs. Register

Unfish features two distinct virtual machine implementations to provide both pedagogical clarity and execution speed:

3.0.1 3.1. Tier 2: 57-Opcode Stack-Based Virtual Machine (`src/vm/uf_vm.c`)

- **Call Frame Structure:**

```
typedef struct {
    UfFunctionObject* fn;
    uint8_t* ip;
    UfValue* slots;
} UfCallFrame;
```

- **Operand Stack:** Contiguous array of 16-byte `UfValue` tagged unions with an explicit `stack_top` pointer.
- **Upvalue Handling:** Open upvalues pointing to stack slots are maintained in a sorted singly-linked list. When a call frame exits, `OP_CLOSE_UPVALUE` copies the value into the heap-allocated upvalue structure, closing the cell.
- **ISA Categories:** Literals, Arithmetic, Bitwise, Control Flow, Closures, Arrays, Maps, Structs, Exceptions, Fibers.

3.0.2 3.2. Tier 3: 256-Register Computed-Goto Virtual Machine (src/vm2/uf_regvm.c)

- **3-Address Instruction Encoding:** Instructions are 32-bit words: [Opcode: 8 bits | Reg_Dest: 8 bits | Reg_Src1: 8 bits | Reg_Src2: 8 bits]
- **Computed-Goto Dispatch:** On GCC and Clang, the execution loop uses direct-threaded labels:

```
#define DISPATCH() goto *dispatch_table[*ip++]
```

This eliminates switch-case jump table overhead and branch mispredictions, speeding up execution by 45%–60%.

- **Register Windowing:** Each call frame receives an independent 256-register window mapped into a flat virtual register file.
-

4 4. Memory Management: Linear Arenas & Mark-Sweep GC

Unfish utilizes a clean, two-phase memory architecture that eliminates manual `free()` and prevents dangling pointers:

4.0.1 4.1. Compilation Linear Arenas (src/common/uf_arena.c)

- AST nodes, tokens, and symbol tables allocated during lexing, parsing, and semantic analysis are bump-allocated inside 64 KB memory chunks.
- When compilation finishes, the entire arena is reclaimed in $O(1)$ time by freeing all linked chunks simultaneously without traversing AST trees.

4.0.2 4.2. Runtime Garbage Collection (src/runtime/uf_runtime.c)

- **Object Layout:** Every heap object (String, Array, Map, Instance, Closure, Buffer) begins with a common header:

```
typedef struct UfObj {
    UfObjType type;
    bool is_marked;
    struct UfObj* next;
} UfObj;
```

- **Root Set Scanning:**
 1. The global environment map.
 2. The VM operand evaluation stack.
 3. All active call frame local variable windows.
 4. The open upvalues linked list.
 5. Suspended fiber stacks in the scheduler queue.
 - **Mark Phase:** Recursively marks reachable objects. Cycles in maps and arrays are safely handled via `is_marked` guard tests.
 - **Sweep Phase:** Walks the linked list of all allocated objects, freeing unmarked nodes and resetting mark bits.
-

5 5. Structured Exception Handling & Unwinding Engine

Unfish implements structured exception handling (`try`, `catch`, `finally`, `raise`, `error()`) across all 5 engines:

```
typedef struct {
    uint8_t* catch_ip;
    size_t frame_index;
    size_t stack_depth;
} UfTryFrame;
```

5.0.1 5.1. The Dual-Stack Synchronization Invariant

- **Handler Registration (OP_PUSH_TRY):** Pushes a `UfTryFrame` recording target catch IP, call frame depth, and operand stack height.
- **Exception Throw (OP_THROW):**
 - Unwinds all call frames created after entering the `try` block (`vm->frame_count = handler->frame_index + 1`).
 - Resets operand stack height to `handler->stack_depth`.
 - Pushes the thrown error value onto the restored stack.
 - Jumps execution directly to `handler->catch_ip`.
- **Early Returns & Loop Breaks:**
 - The compiler tracks `try_depth` statically. When a `return`, `break`, or `continue` escapes an active `try` block, it emits explicit `OP_POP_TRY` / `ROP_POP_TRY` instructions before jumping, preventing handler stack desynchronization.

6 6. Systems Programming & Low-Level Buffers

Unfish bridges high-level scripting with low-level systems programming through built-in byte buffers:

```
let packet = buffer_create(32)
buffer_write_u32(packet, 0, 0xDEADBEEF, true) # 32-bit Little-Endian Magic
buffer_write_u16(packet, 4, 1024, true)       # 16-bit Payload Length
buffer_write_f64(packet, 6, 26.5, true)       # 64-bit IEEE 754 Float
```

6.0.1 6.1. Buffer Capabilities

- **Explicit Endianness:** All 16-bit, 32-bit, and 64-bit reads and writes support explicit little-endian and big-endian flags.
- **Bitwise Manipulation:** Standard 32-bit two's complement bitwise logic (`band`, `bor`, `bxor`, `bnot`, `shl`, `shr`, `sar`).
- **Memory Safety:** Every buffer read and write enforces strict bounds checking, raising catchable exceptions on out-of-bounds access.

7 7. Ahead-of-Time Native C99 Transpiler & WebAssembly

7.0.1 7.1. C99 Code Generation (`src/codegen/uf_emit_c.c`)

- Compiles Unfish scripts directly to human-readable ANSI C99 source code.
- Links against the standalone, single-header runtime engine `src/codegen/unfish_runtime.h`.
- Can be compiled with `gcc -O3` or `clang -O3` into standalone native binaries with zero external dependencies.

7.0.2 7.2. WebAssembly Target (`--wasm`)

- Emits WebAssembly bytecode targeting the standard WebAssembly System Interface (WASI).

- Runs inside web browsers, Node.js, and WASI runtimes (`wasmtime`, `wasmer`) with identical output.

7.0.3 7.3. Bare-Metal Embedded Target (`--embedded --arm`)

- Generates freestanding C99 code (`-DUF_EMBEDDED`) without operating system dependencies.
- Runs directly on ARM Cortex-M microcontrollers (STM32, Raspberry Pi Pico, nRF52) with memory-mapped I/O.

8 8. Verification & Differential Parity Harness

Every commit is verified by the **Five-Way Differential Parity Harness** (`tools/run_differential_tests.sh`):

Differential Test Results: 95 passed (identical), 0 diverged.

- **95 Conformance Tests:** Covering arithmetic, closures, loops, exceptions, structs, maps, modules, f-strings, and systems buffers.
- **100% Behavioral Identity:** `stdout`, `stderr`, and exit codes must match across AST Interpreter, Stack VM, Register VM, Native C99, and WebAssembly.
- **Sanitizer Verification:** `make test-asan` verifies clean AddressSanitizer and LeakSanitizer runs (0 leaks, 0 errors).